

Technical Report - Assignment 04

2317022 이다영

개요

이미지 복원(Image Restoration)은 noise, out-of-focus blur, motion blur 등의 degradation이 적용된 이미지에서 원본 이미지를 복원하는 작업이다. 손상되지 않은 목표 이미지 $f(x,y)$ 에 blurring 커널 $h(x,y)$ 와 노이즈 $n(x,y)$ 가 작용한 손상 이미지 $g(x,y)$ 로부터 $f(x,y)$ 를 찾는 것으로, 수식으로 나타내면 아래와 같다.

$$g(x,y) = h(x,y) * f(x,y) + n(x,y)$$

본 과제에서는 Salt & Pepper 노이즈, Gaussian 노이즈를 각각 이미지에 추가하고, Median filter, Gaussian filter, Bilateral filter를 이용하여 노이즈를 제거하는 세 가지 프로그램을 구현하였다. 경계 처리 방식으로는 zero-padding, mirroring, adjustkernel 세 가지를 모두 구현하였으며, gray-scale, 컬러(RGB) 이미지 모두를 지원한다.

참고로 결과 스크린샷과 분석은 각 파일에 대한 설명을 모두 한 뒤에 마지막 부분에서 한꺼번에 분석한다.

salt_and_pepper.cpp

Salt and pepper 노이즈는 이미지에 흰색(salt) 또는 검은색(pepper) 점이 무작위로 나타나는 임펄스 노이즈이다. 값이 0인 픽셀을 pepper 노이즈, 값이 255인 픽셀을 salt 노이즈라 한다.

이를 제거하기 위해 미디언(median) 필터를 사용한다. 커널 내 픽셀 값을 정렬한 후 중앙값을 레퍼런스 픽셀에 할당하는 방식으로, 극단적인 값을 가진 노이즈 픽셀을 효과적으로 제거한다. uniform mean 필터가 모든 픽셀에 균등 가중치를 적용해 outlier 값에 민감하고 이미지를 blurry하게 만드는 것과 달리, 미디언 필터는 중앙값 선택으로 outlier를 배제하므로 노이즈 제거 효과가 뛰어나다.

1. main()

main() 함수는 lena.jpg를 읽어 grayscale로 변환한 후, 노이즈 추가 함수와 노이즈 제거 함수를 순서대로 호출하여 결과를 출력 창에 표시한다.

사용하는 주요 Mat 객체:

- input: 입력 컬러 이미지
- input_gray: input을 그레이스케일로 변환한 이미지
- noise_Gray: input_gray에 salt & pepper 노이즈를 추가한 이미지
- noise_RGB: input에 salt & pepper 노이즈를 추가한 이미지
- Denoised_Gray: noise_Gray에서 미디언 필터로 노이즈를 제거한 grayscale 이미지
- Denoised_RGB: noise_RGB에서 미디언 필터로 노이즈를 제거한 컬러 이미지

2. Mat Add_salt_pepper_Noise(const Mat input, float ps, float pp)

입력 이미지에 salt & pepper 노이즈를 생성하여 반환하는 함수이다. ps는 salt 노이즈 비율, pp는 pepper 노이즈 비율이며, 전체 픽셀 수에 각 비율을 곱하여 노이즈를 추가할 픽셀 수를 결정한다.

```
int amount1 = (int)(output.rows * output.cols * pp); // pepper 픽셀 수
int amount2 = (int)(output.rows * output.cols * ps); // salt 픽셀 수
```

그레이스케일 이미지(채널 1개)의 경우 무작위 픽셀에 0(pepper) 또는 255(salt)를 직접 할당한다. 컬러 이미지(채널 3개)는 각 RGB 채널별로 독립적으로 무작위 픽셀을 선택하여 0 또는 255를 할당하므로, 세 채널 모두 동시에 변경되는 경우가 아니라면 검은색/흰색이 아닌 색깔 있는 노이즈가 생성될 수 있다.

3. Mat Salt_pepper_noise_removal_Gray(const Mat input, int n, const char* opt)

그레이스케일 이미지에 미디언 필터를 적용하여 salt & pepper 노이즈를 제거하는 함수이다.

3.1 변수 및 커널 초기화

kernel_size = 2n+1로 (kernel_size * kernel_size) x 1 크기의 커널 Mat 객체를 생성한다. 각 픽셀 (i, j)마다 이전 픽셀의 값이 남지 않도록 루프 내부에서 커널을 재초기화한다.

```
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        kernel = Mat::zeros(kernel_size * kernel_size, 1, input.type()); // 매 픽셀마다 재초기화
        std::vector<G> kernel_vector = {}; // adjustkernel용
```

zero-padding 방식에서 경계 밖 픽셀은 kernel을 0으로 초기화한 상태로 두어 zero-padding 효과를 구현한다. kernel 재초기화를 루프 안에서 수행하지 않으면 이전 픽셀의 값이 잔류하여 오

류가 발생한다.

3.2 Zero-padding

커널 좌표 $(i+x, j+y)$ 가 이미지 범위 내이면 해당 픽셀 값을 kernel에 저장하고, kernel_index를 매 반복마다 증가시킨다. 범위 밖인 경우 아무것도 하지 않으므로, 초기화된 0 값이 그대로 유지되어 zero-padding이 구현된다.

```
if ((i+x <= row-1) && (i+x >= 0) && (j+y <= col-1) && (j+y >= 0)) {  
    kernel.at<G>(kernel_index, 0) = input.at<G>(i+x, j+y);  
}  
kernel_index++; // 범위 밖에서도 인덱스 증가
```

3.3 Mirroring

커널 좌표가 이미지 범위를 벗어날 경우 아래 공식으로 반사된 인덱스를 계산한다.

```
// 행(i+x)이 범위 초과 시  
if (i+x > row-1) tempa = 2*(row-1) - (i+x); // 오른쪽 경계 반사  
if (i+x < 0) tempa = -(i+x); // 왼쪽 경계 반사  
// 열(j+y)이 범위 초과 시  
if (j+y > col-1) tempb = 2*(col-1) - (j+y); // 하단 경계 반사  
if (j+y < 0) tempb = -(j+y); // 상단 경계 반사
```

3.4 Adjustkernel

이미지 범위 내의 유효한 픽셀만 kernel_vector에 push_back()으로 추가하여, 경계 부근에서 유효 픽셀 수가 달라지는 것을 동적으로 처리한다.

```
if ((i+x <= row-1) && (i+x >= 0) && (j+y <= col-1) && (j+y >= 0)) {  
    kernel_vector.push_back(input.at<G>(i+x, j+y));  
}
```

3.5 커널 정렬 및 중앙값 계산

adjustkernel의 경우 std::sort로 kernel_vector를 정렬하고, 벡터 크기의 절반 인덱스를 중앙값 인덱스로 사용한다. 그 외 옵션은 OpenCV의 sort()로 kernel Mat을 열 단위 오름차순 정렬한 후, $(\text{kernel_size} * \text{kernel_size} / 2)$ 인덱스의 값을 중앙값으로 결과 이미지에 할당한다.

4. Mat Salt_pepper_noise_removal_RGB(const Mat input, int n, const char* opt)

컬러 이미지에 미디언 필터를 적용한다. 기본 구조는 그레이스케일 버전과 동일하며, 채널 처리 방식에 차이가 있다.

커널은 (kernel_size * kernel_size) x 3 크기의 3열짜리 Mat으로 생성하며(각 열이 R, G, B 채널에 해당), 타입은 input.type() - 16을 사용하여 3채널 타입을 단채널 타입으로 변환한다 (CV_8UC3 - CV_8U = 16).

adjustkernel 옵션에서는 채널별로 독립적인 kernel_vector_r, kernel_vector_g, kernel_vector_b 벡터를 사용한다. 정렬 후 중앙값을 결과 이미지의 각 채널에 각각 할당한다.

```
kernel.at<G>(kernel_index, 0) = input.at<C>(i+x, j+y)[0]; // R 채널
kernel.at<G>(kernel_index, 1) = input.at<C>(i+x, j+y)[1]; // G 채널
kernel.at<G>(kernel_index, 2) = input.at<C>(i+x, j+y)[2]; // B 채널
```

Gaussian.cpp

가우시안 노이즈는 픽셀 값에 평균 0, 표준편차 sigma의 가우시안 분포를 따르는 랜덤 값이 더해진 형태이다. 수식으로는 $I(x,y) = I(x,y) + N(x,y)$ 로 표현된다.

가우시안 노이즈 제거를 위해 가우시안 필터를 사용한다. 레퍼런스 픽셀로부터의 거리에 가우시안 함수를 적용한 가중치로 주변 픽셀들을 평균하여, 단순 산술 평균인 uniform mean 필터보다 향상된 필터링 성능을 제공한다.

1. main()

main() 함수에서 8비트 이미지를 CV_64F(64비트 부동소수점)로 변환하여 [0,1] 범위로 정규화한 후 처리한다.

```
input.convertTo(input, CV_64FC3, 1.0 / 255);
input_gray.convertTo(input_gray, CV_64F, 1.0 / 255);
```

이후 Add_Gaussian_noise()로 노이즈를 추가하고, Gaussianfilter_Gray() / Gaussianfilter_RGB()로 노이즈를 제거한 결과를 출력 창에 표시한다.

2. Mat Add_Gaussian_noise(const Mat input, double mean, double sigma)

OpenCV의 RNG(Random Number Generator)와 rng.fill()을 사용하여 지정된 mean과 sigma의 가우시안 분포를 따르는 노이즈 배열 NoiseArr를 생성하고, add() 함수로 원본 이미지에 더한

다.

```
Mat NoiseArr = Mat::zeros(input.rows, input.cols, input.type());
RNG rng;
rng.fill(NoiseArr, RNG::NORMAL, mean, sigma);
add(input, NoiseArr, NoiseArr);
```

3. Mat Gaussianfilter_Gray(const Mat input, int n, float sigmaT, float sigmaS, const char* opt)

3.1 가우시안 커널 초기화

(2n+1) x (2n+1) 크기의 가우시안 커널을 생성한다. 각 위치 (a, b)의 가중치는 아래 공식으로 계산한 후 전체 합(denom)으로 나눠 정규화한다.

$$\text{kernel}(a,b) = \exp(-a^2 / (2 \cdot \text{sigmaS}^2) - b^2 / (2 \cdot \text{sigmaT}^2))$$

```
float value1 = exp(-(pow(a,2)/(2*pow(sigmaS,2))) - (pow(b,2)/(2*pow(sigmaT,2))));
kernel.at<float>(a+n, b+n) = value1;
denom += value1;
// 이후 denom으로 나눠 정규화
kernel.at<float>(a+n, b+n) /= denom;
```

3.2 Zero-padding

커널 위치 (i+a, j+b)가 이미지 범위 내인 경우에만 가중치 x 픽셀 값을 sum1에 누적한다. 범위 밖은 0으로 처리된다.

```
if ((i+a <= row-1) && (i+a >= 0) && (j+b <= col-1) && (j+b >= 0)) {
    sum1 += kernel.at<float>(a+n, b+n) * input.at<G>(i+a, j+b);
}
```

3.3 Mirroring

경계를 벗어나는 좌표를 반사 공식으로 변환한다. 양수 방향 경계 초과 시 $2 * (\text{max_idx}) - \text{coord}$ 공식을 사용하고, 음수 방향은 절댓값으로 반사한다.

```
if (i+a > row-1) tempa = 2*(row-1) - (i+a);
if (i+a < 0)    tempa = -(i+a);
if (j+b > col-1) tempb = 2*(col-1) - (j+b);
if (j+b < 0)    tempb = -(j+b);
```

```
sum1 += kernel.at<float>(a+n, b+n) * input.at<G>(tempa, tempb);
```

3.4 Adjustkernel

유효한 픽셀만 sum1에 누적하고, 해당 픽셀의 커널 가중치도 sum2에 누적한다. 최종적으로 sum1/sum2로 나눠 실제 사용된 가중치 합으로 재정규화한다.

```
if ((i+a <= row-1) && (i+a >= 0) && (j+b <= col-1) && (j+b >= 0)) {  
    sum1 += kernel.at<float>(a+n, b+n) * input.at<G>(i+a, j+b);  
    sum2 += kernel.at<float>(a+n, b+n);  
}  
output.at<G>(i,j) = (G)(sum1 / sum2);
```

4. Mat Gaussianfilter_RGB(const Mat input, int n, double sigmaT, double sigmaS, const char* opt)

가우시안 커널 생성과 경계 처리 로직은 그레이스케일 버전과 동일하다. 컬러 이미지이므로 각 채널(R, G, B)에 대해 독립적인 누적합 변수(sum1_r, sum1_g, sum1_b)를 사용하여 채널별로 가우시안 필터를 적용한다.

```
sum1_r += kernel.at<float>(a+n, b+n) * (float)(input.at<C>(i+a, j+b)[0]);  
sum1_g += kernel.at<float>(a+n, b+n) * (float)(input.at<C>(i+a, j+b)[1]);  
sum1_b += kernel.at<float>(a+n, b+n) * (float)(input.at<C>(i+a, j+b)[2]);  
output.at<C>(i,j)[0] = (G)sum1_r;  
output.at<C>(i,j)[1] = (G)sum1_g;  
output.at<C>(i,j)[2] = (G)sum1_b;
```

Bilateral.cpp

가우시안 필터는 레퍼런스 픽셀로부터의 공간적 거리만을 고려하여 가중치를 계산하므로, 에지 부근 픽셀도 거리가 가까우면 큰 가중치가 적용되어 에지가 희석되고 이미지가 blurry해진다.

Bilateral 필터는 공간적 거리(spatial difference, G_s)와 픽셀 값 차이(range difference, G_r)를 모두 고려한다. 레퍼런스 픽셀과 intensity가 크게 다른 픽셀(에지)에는 작은 가중치를 부여하여 에지를 보존한다.

$$Op = (1/Wp) \cdot \sum Gs(|p-q|) \cdot Gr(|Ip-Iq|) \cdot Iq$$

$$W_p = \sum G_s(|p-q|) \cdot Gr(|I_p-I_q|)$$

공간 커널 G_s 는 거리가 가까울수록, intensity 커널 Gr 은 밝기 차이가 작을수록 큰 가중치를 부여한다.

1. main()

Gaussian.cpp와 동일하게 이미지를 CV_64F로 변환한 후 Add_Gaussian_noise()로 노이즈를 추가하고, bilateralfilter_Gray() / bilateralfilter_RGB()로 bilateral 필터링을 수행하여 결과를 출력한다.

2. Mat bilateralfilter_Gray(const Mat input, int n, float sigmaT, float sigmaS, float sigmaR, const char* opt)

2.1 공간 커널 G_s 초기화

가우시안 필터의 커널 초기화와 동일한 방식으로 G_s 를 생성하고 정규화한다. Bilateral 필터에서는 이 커널이 공간 커널로 사용되며, 이후 intensity 커널 Gr 과 함께 사용된다.

```
float value1 = exp(-(pow(a,2)/(2*pow(sigmaS,2))) - (pow(b,2)/(2*pow(sigmaT,2))));
Gs.at<float>(a+n, b+n) = value1;
```

2.2 Zero-padding

이미지 범위 내의 픽셀에 대해 intensity 커널 Gr 을 계산하고, $G_sGr = G_s * Gr$ 을 구하여 sum1과 W 를 갱신한다. 범위 밖 픽셀의 경우 Gr 이 픽셀 값에 의존하므로 계산할 수 없어 sum1과 W 모두 갱신하지 않는다.

```
if ((i+a <= row-1) && (i+a >= 0) && (j+b <= col-1) && (j+b >= 0)) {
    Gr = exp(-(pow(input.at<G>(i,j) - input.at<G>(i+a,j+b), 2) / (2*pow(sigmaR,2))));
    GsGr = Gs.at<float>(a+n, b+n) * Gr;
    sum1 += GsGr * input.at<G>(i+a, j+b);
    W += GsGr; // if 블록 안에서 갱신 (경계 밖은 기여하지 않음)
}
```

가우시안 필터의 zero-padding에서는 경계 밖 픽셀 값이 0으로 처리되어 sum에 더해도 결과가 같으므로 범위 밖에서도 인덱스만 증가하면 됐다. 반면 bilateral 필터는 W 가 분모로 사용되므로 경계 밖 픽셀이 W 에 포함될 경우 출력 값이 왜곡된다. 따라서 W 갱신을 if 블록 안에 위치시킨다.

2.3 Mirroring

Gaussian.cpp와 동일한 반사 공식으로 경계 밖 좌표를 변환한다. 반사된 좌표 (tempa, tempb)로 Gr을 계산하며, 모든 커널 위치에 대해 GsGr과 W를 계산한다.

```
if (i+a > row-1) tempa = 2*(row-1) - (i+a);
if (j+b > col-1) tempb = 2*(col-1) - (j+b);
float Gr = exp(-(pow(input.at<G>(i,j) - input.at<G>(tempa,tempb), 2) / (2*pow(sigmaR,2))));
W += GsGr; // mirroring은 모든 위치에서 유효한 픽셀을 사용하므로 항상 갱신
```

2.4 Adjustkernel

유효 픽셀에 대해서만 Gr, GsGr, sum1, W를 계산하여 경계 픽셀의 부족한 이웃을 재정규화로 보정한다.

2.5 최종 픽셀 값 계산

W가 0이 아닌 경우 sum1/W로 출력 픽셀 값을 계산한다. divide-by-zero를 방지하기 위해 W == 0인 경우 sum1을 그대로 사용한다.

```
if (W != 0) output.at<G>(i,j) = (G)(sum1 / W);
else      output.at<G>(i,j) = (G)sum1;
```

3. Mat bilateralfilter_RGB(const Mat input, int n, float sigmaT, float sigmaS, float sigmaR, const char* opt)

컬러 이미지에 bilateral 필터를 적용할 때 intensity 커널 Gr을 계산하기 위해 각 픽셀의 컬러 값을 3x1 열벡터 Cp, Cq로 표현한다.

```
Mat Cp = Mat::zeros(3, 1, CV_32F);
Cp.at<float>(0,0) = input.at<C>(i,j)[0]; // R
Cp.at<float>(1,0) = input.at<C>(i,j)[1]; // G
Cp.at<float>(2,0) = input.at<C>(i,j)[2]; // B
```

intensity 커널 Gr은 Cp와 Cq의 색상 차이 벡터 $d = C_p - C_q$ 의 제곱합($d \cdot d$)을 이용해 계산한다.

```
Mat d = Cp - Cq;
float powd = d.dot(d); // |Cp - Cq|^2 (컬러 거리의 제곱)
float Gr = exp(-powd / (2*pow(sigmaR,2)));
```

누적합 sum1도 스칼라가 아닌 3x1 Mat 벡터로 선언하며, 최종 출력은 $Mat Op = sum1/W$ 로

계산한 후 각 채널에 할당한다.

```
Mat sum1 = Mat::zeros(3, 1, CV_32F);
sum1 += GsGr * Cq;
// ...
Mat Op = sum1 / W;
output.at<C>(i,j)[0] = Op.at<float>(0,0);
output.at<C>(i,j)[1] = Op.at<float>(1,0);
output.at<C>(i,j)[2] = Op.at<float>(2,0);
```

결과 분석

입력 이미지로 lena.jpg를 사용하였다. 모든 실험에서 그레이스케일과 컬러 이미지를 모두 처리하며, 경계 처리 옵션 비교 시 zero-padding, mirroring, adjustkernel을 각각 적용한 결과를 확인하였다.

1. Salt & Pepper 노이즈 추가 및 제거

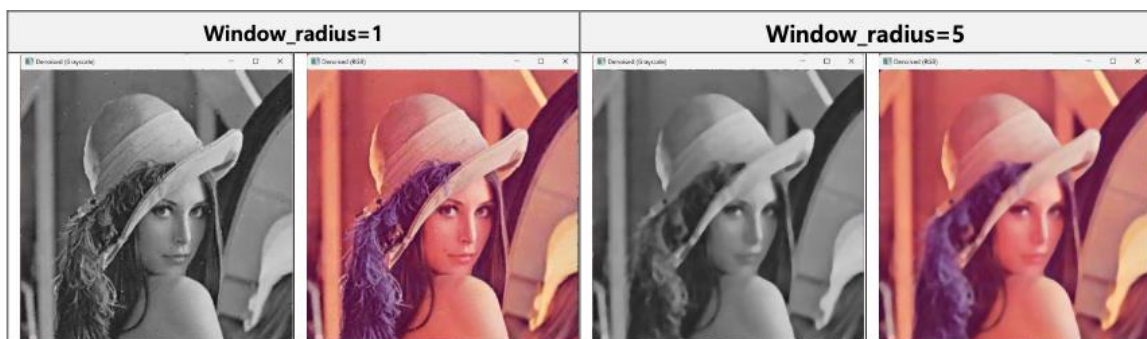
ps=0.1, pp=0.1(각 10%)로 노이즈를 추가하고, n=2(커널 크기 5x5), zero-padding으로 미디언 필터를 적용하였다.

[위: Salt and pepper noise 추가, 아래: Salt and pepper noise 제거]

노이즈를 추가한 위의 결과 사진의 경우, grayscale 노이즈 이미지에는 흰색과 검은색의 노이즈가 표시되고 RGB 노이즈 이미지에는 색깔이 있는 노이즈가 표시되는데, 이는 Add_salt_pepper_Noise() 함수에서 컬러 이미지에 노이즈를 생성할 때 한채널만을 변경하여 전체 채널이 0 또는 255 가 되지 않는 경우가 존재하기 때문이다.

노이즈를 제거한 아래 결과의 경우, grayscale, 컬러 이미지 모두 커널 반지름을 2로 "zero padding" 옵션을 전달해 노이즈 제거 함수를 호출, 크기가 $(2*2+1=5)$ 인 커널을 사용해 zero padding 방식으로 경계 처리를 수행하여 salt and pepper 노이즈를 제거했다. 그 결과 엣지가 다소 흐려졌지만 노이즈가 제거된 것을 볼 수 있었다.

2. 커널 크기별 비교 (Salt & Pepper)



n=1과 n=5로 비교한 결과:

- n=1: 작은 커널로 주변 픽셀을 충분히 고려하지 못해 일부 노이즈가 제거되지 않음. 엣지는 상대적으로 선명.
- n=5: 넓은 범위에서 중앙값을 선택하므로 노이즈 제거 효과는 우수하나, 세부 정보가 희석됨.

커널 크기가 너무 작으면 노이즈 제거 성능이 저하되고, 너무 크면 엣지 및 세부 정보가 손실되므로 적절한 크기 선택이 중요하다.

3. 가우시안 노이즈 추가 및 제거



[위: gaussian noise 추가, 아래: gaussian noise 제거]

mean=0, sigma=0.1로 노이즈를 추가하고, n=5, sigmaT=sigmaS=3, zero-padding으로 가우시안 필터를 적용하였다.

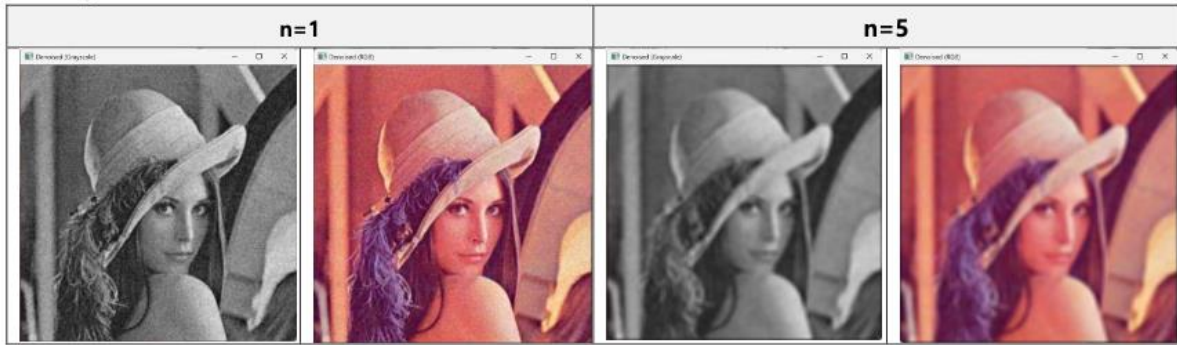
Salt & pepper 노이즈와 달리 가우시안 노이즈는 다양한 intensity 값으로 나타나 이미지가 고르게 흐려 보인다. 가우시안 필터 적용 후 노이즈는 감소하나 커널 내 값들을 가중 평균하는 방식으로 인해 이미지가 다소 blurry해진다.

sigma=0.5로 설정 시 분포 폭이 넓어져 더 강한 노이즈가 생성되며, 제거 후에도 sigma=0.1에 비해 복원 품질이 낮아진다.

4. 커널 크기별 비교 (가우시안 필터)

- n=1: 노이즈 제거 효과가 미미하고 에지는 잘 보존됨.

- $n=5$: 노이즈 제거 효과는 우수하나 결과 이미지가 더욱 blurry해져 세부 정보 소실.



5. Bilateral 필터 결과 및 가우시안 필터 비교

동일 조건($n=5$, $\sigma_T=\sigma_S=3$)으로 가우시안 필터와 bilateral 필터를 비교하였다. bilateral 필터의 $\sigma_R=0.2$ 로 설정.

가우시안 필터는 공간적 거리만 고려하므로 에지 근처 픽셀도 동등하게 평균되어 에지가 희석된다. 반면 bilateral 필터는 intensity 차이도 고려하여 에지를 사이에 둔 픽셀에 작은 가중치를 부여하므로 결과 이미지의 에지가 선명하게 보존된다.

σ_R 값이 작을수록 intensity 커널의 영향이 커져 에지 보존 효과가 강해지며, 값이 클수록 가우시안 필터에 가까운 결과를 보인다.



6. 경계 처리 옵션 비교

세 가지 경계 처리 옵션(zero-padding, mirroring, adjustkernel)을 동일 파라미터로 비교한 결과, 이미지 중심부에서는 모든 옵션의 결과가 거의 동일하다. 이미지 경계 부근에서 차이가 나타나는데:

- zero-padding: 경계 밖을 0으로 처리하므로 이미지 가장자리가 어둡게 처리될 수 있음.
- mirroring: 경계 픽셀을 반사하여 사용하므로 자연스러운 경계 처리가 가능하며, 인위적인 경계 효과가 줄어듦.

- adjustkernel: 유효 픽셀만으로 재정규화하여 계산하므로 경계 픽셀의 가중치 합이 보정됨.
-

결론

본 과제에서는 세 가지 노이즈 제거 알고리즘을 구현하고 비교하였다.

- 미디언 필터: Salt & pepper 노이즈 제거에 효과적이며, 에지 보존 성능이 우수하다. 극단적 outlier 값을 중앙값 선택으로 효과적으로 배제한다.
- 가우시안 필터: 가우시안 노이즈 제거에 적합하나, 공간적 거리만 고려하므로 이미지가 다소 blurry해진다.
- Bilateral 필터: 공간적 거리와 intensity 차이를 모두 고려하여 에지를 보존하면서 노이즈를 제거한다. 가우시안 필터 대비 에지 보존 성능이 뛰어나다.

커널 크기와 파라미터(sigma 등)는 노이즈 제거 성능과 엣지/세부 정보 보존 간의 트레이드오프에 직접 영향을 미치므로, 이미지와 노이즈의 특성에 맞게 적절히 조정해야 함을 알게 되었다.